

FACE RECOGNITION

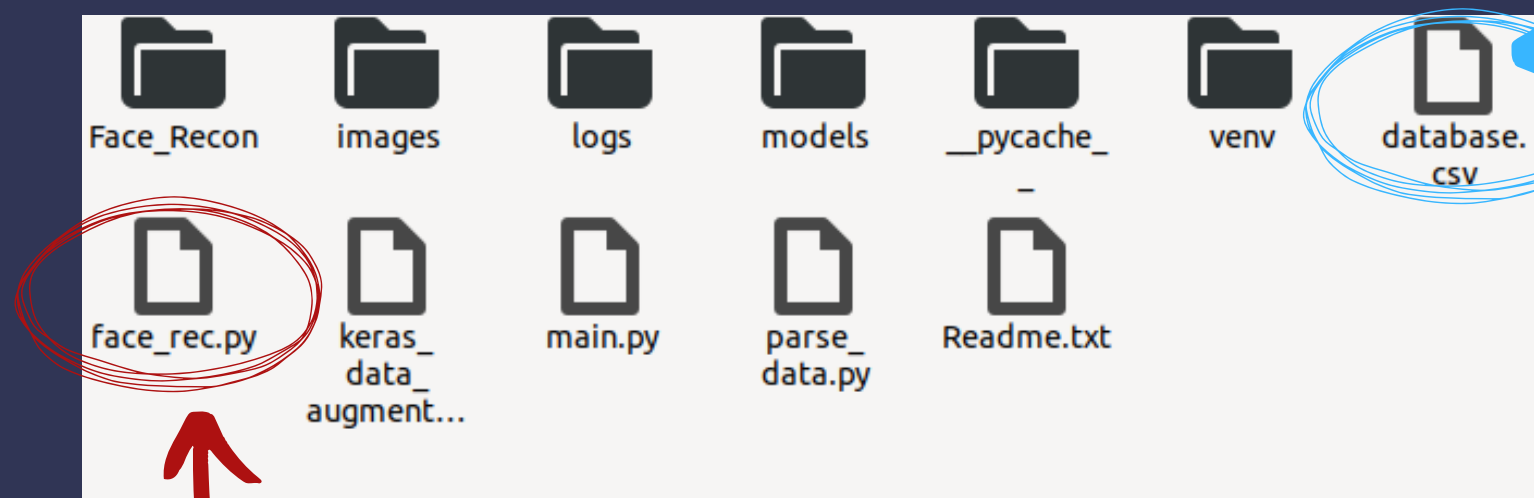
Big data analytics



Live Face Recognition



Application Folder



columns : Name, Birthday, Age, Profession, MAC, Height, Weight, Eye, Gender, Country, File_name, Bureau Record

B	C	D	E	F	G	H	I	J
Name	Birthday	Age	Profession	MAC	Height	Weight	Eye	Eye_abbreviation
HEDI RAGAD	28/05/1982.	39	Founder & Director iTeam	AC:15:D0:85:G3:F81	175.2	78.4	Brown	BR
IRINA PEKAR	22/02/1964.	58	Hydrolic Engineer	AC:15:D0:85:G3:F81	165.8	65.8	Green	GR
INES KHELIFI	01/04/1990.	32	Developer	AC:15:D0:85:G3:F81	168.7	57.8	Brown	BR

```
import cv2
import dlib
import face_recognition
import parse_data
from PIL import Image, ImageDraw
import subprocess
from datetime import datetime
```

Main Libraries : cv2 and face_recognition

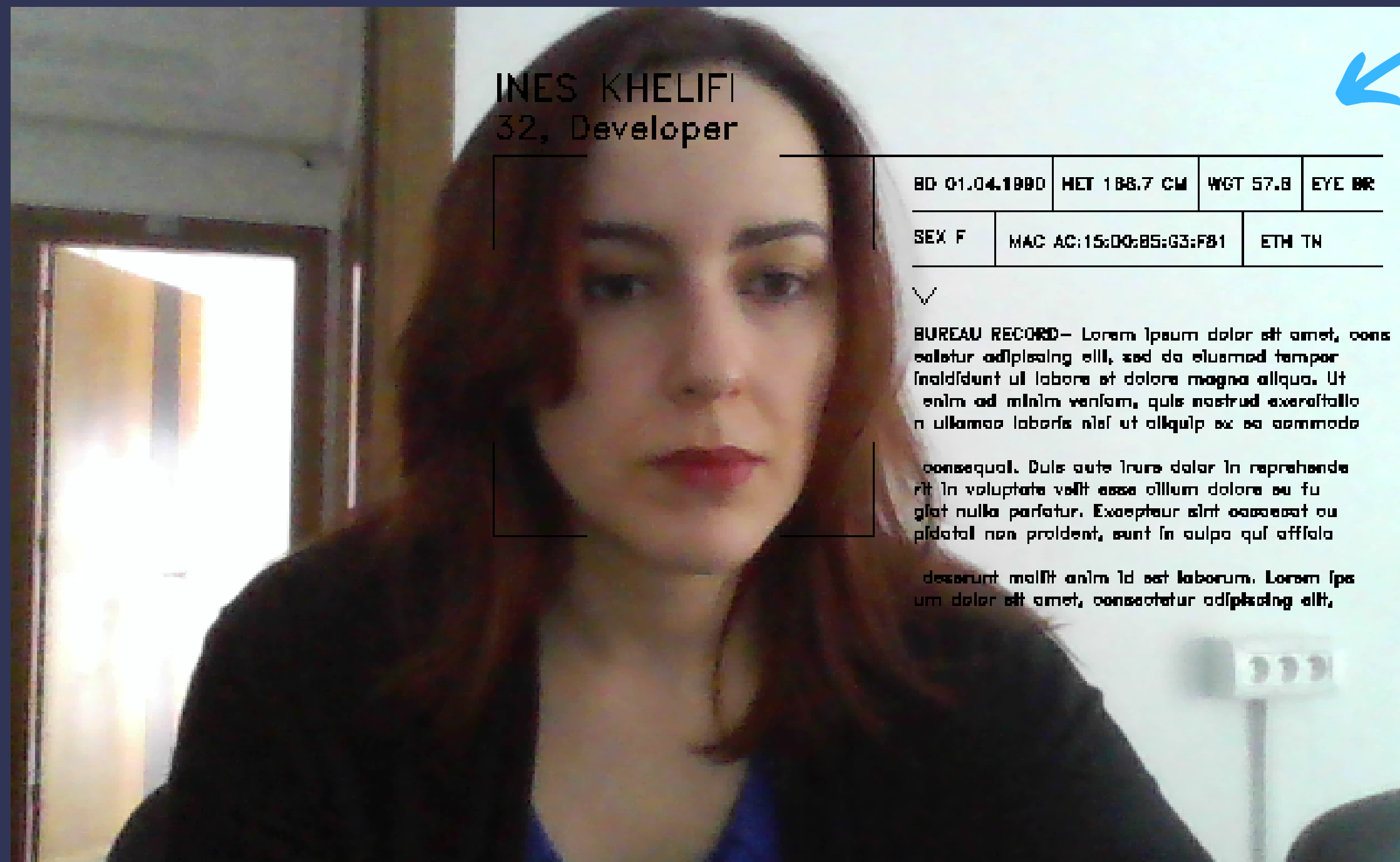
```
while True:
    ret,current_frame = webcam_video_stream.read()
    current_frame_small = cv2.resize(current_frame,(0,0),fx=0.25,fy=0.25)
    #detect all faces in the image
    #arguments are image,no_of_times_to_upsample, model
    all_face_locations = face_recognition.face_locations(current_frame_small,number_of_times_to_
mple=1,model='hog')
```

```
# Display the name as text in the image
font1 = cv2.FONT_HERSHEY_DUPLEX
font2 = cv2.FONT_HERSHEY_TRIPLEX
```

```
cv2.line(imge, (left_pos,top_pos), (right_pos-k, top_pos), color_frame, thi
cv2.line(imge, (right_pos,top_pos), (left_pos+k, top_pos), color_frame, thi
cv2.line(imge, (left_pos,bottom_pos), (right_pos-k, bottom_pos), color_fram
cv2.line(imge, (right_pos,bottom_pos), (left_pos+k, bottom_pos), color_fram
```

- 1- **Capture live video** : cv2.VideoCapture(0)
- 2- **Loop** through every frame using **face_recognition** library to identify faces in the frame.
- 3- **Frame Design** using cv2.line() and cv2.rectangle() functions
- 4- For every person **identified** in the frame, save the picture using pil_image.save()

Face recognition

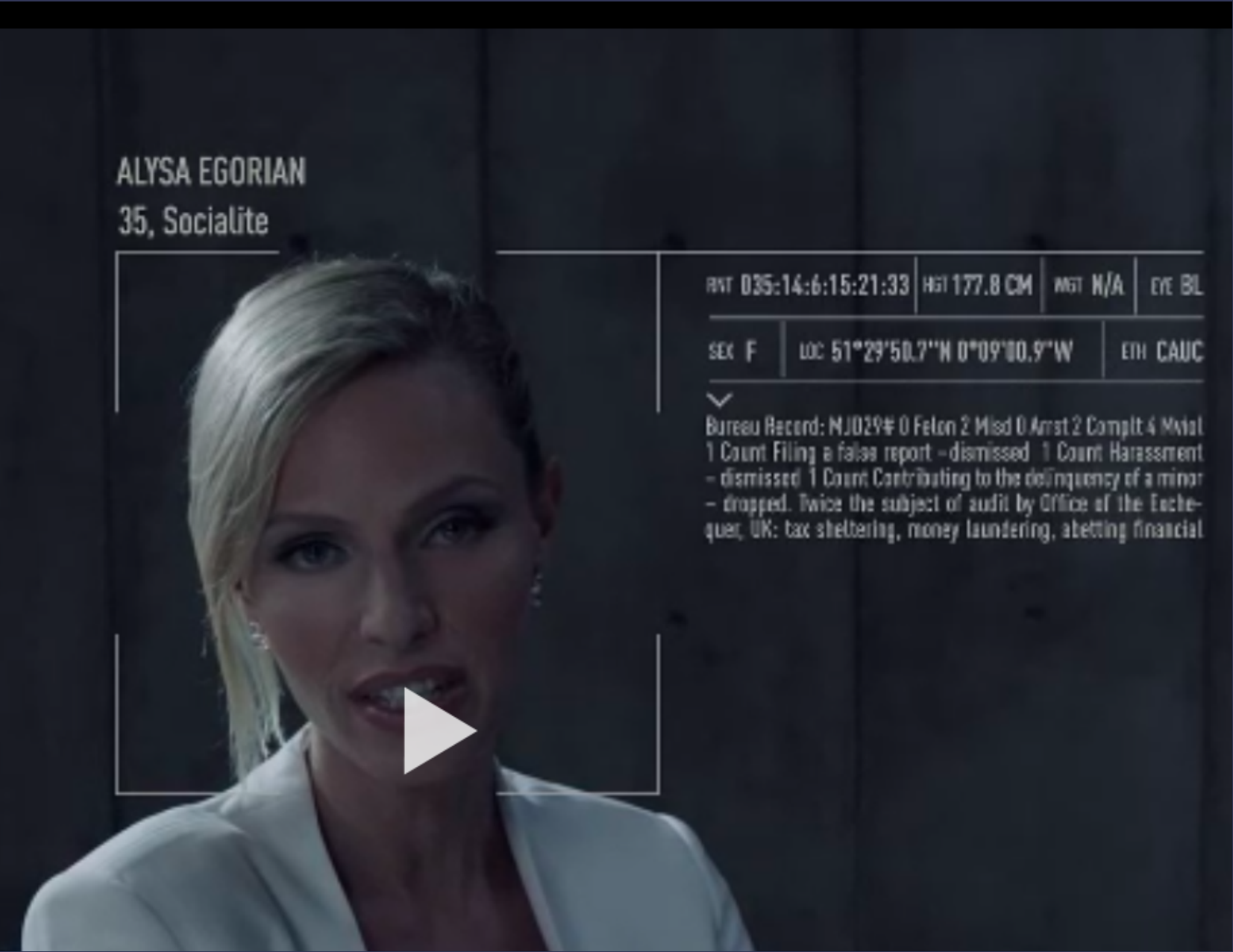


Informations :

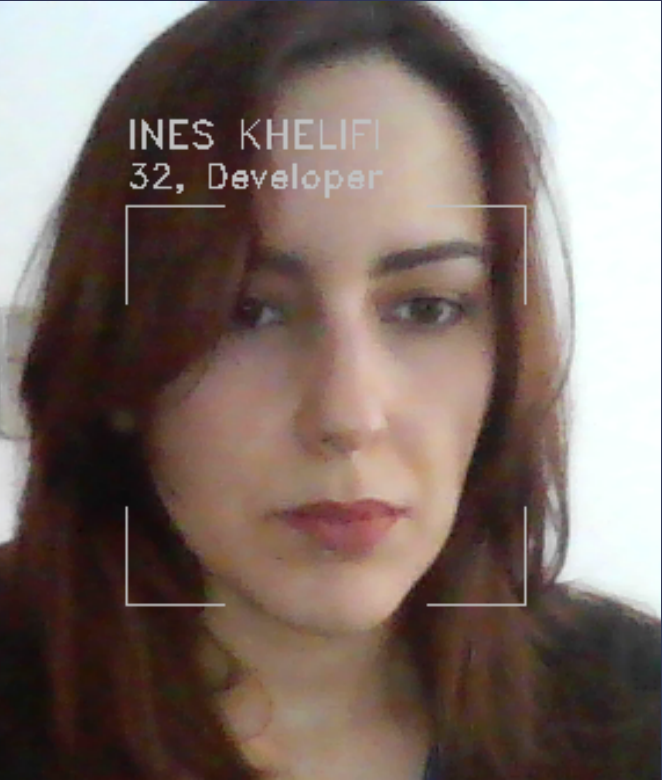
- **Title** : Name, age, profession
- **Extended** : birthday, height, weight, eye color, gender, MAC, ethnicity, bureau record.

Face recognition

Extended info bar

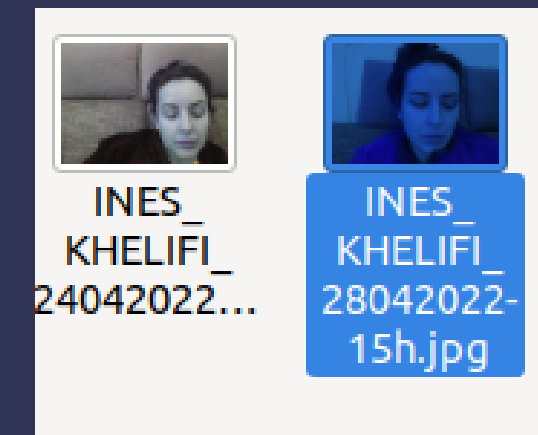
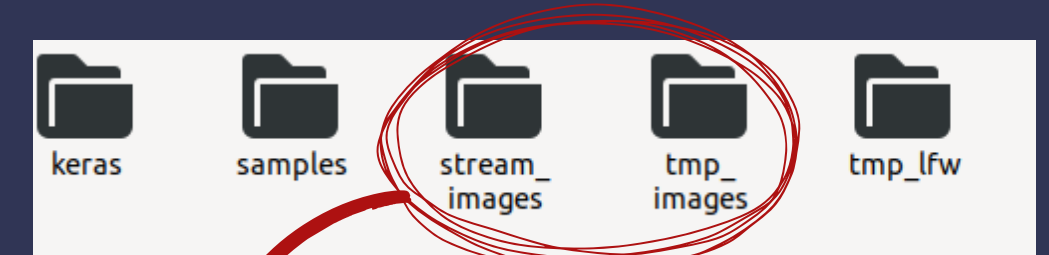
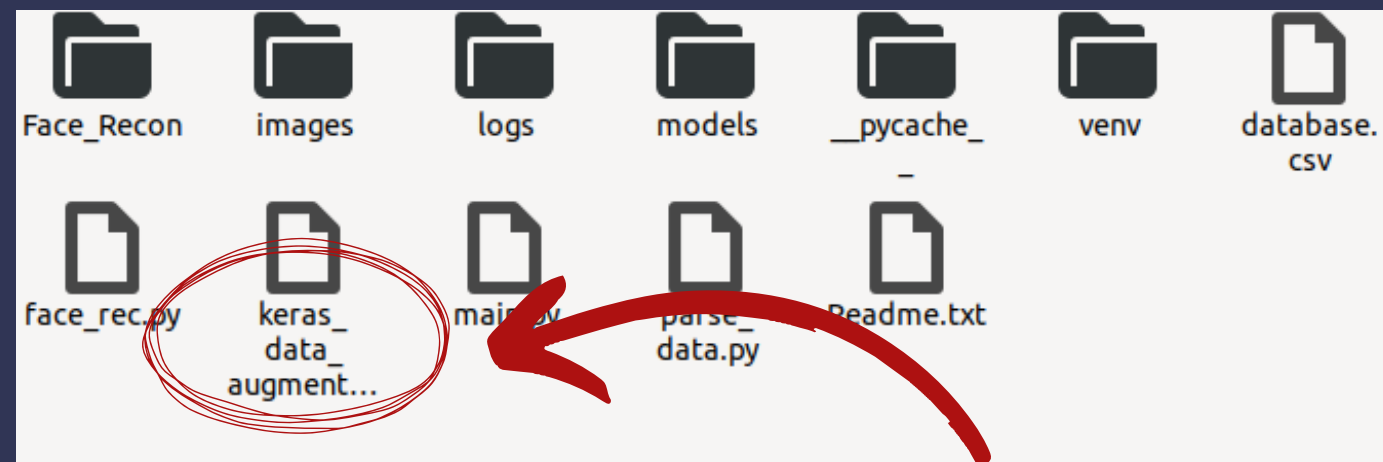


Minimal info bar

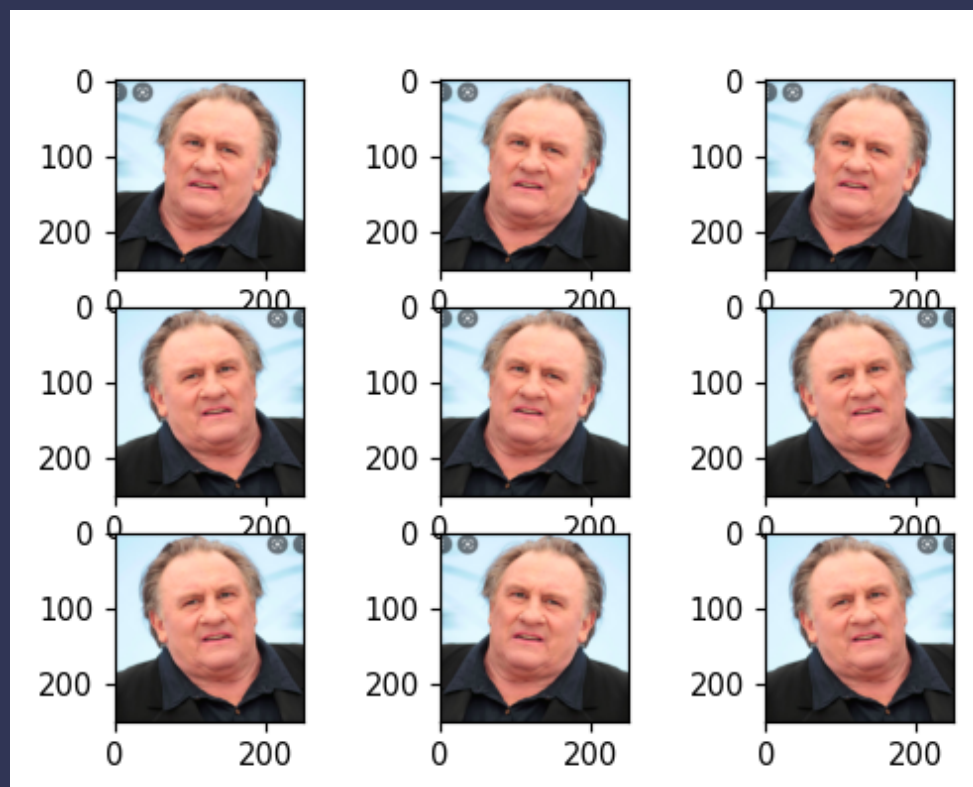


Live Images

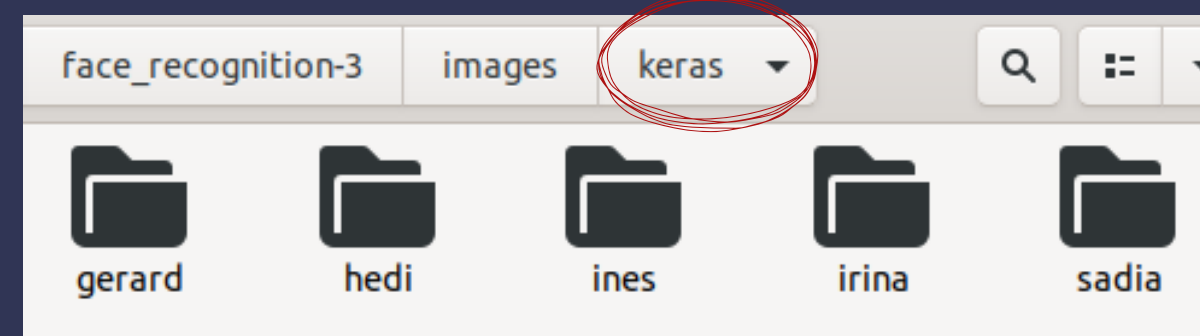
Data Augmentation with keras



Identified pictures in live Camera are **saved** in images/tmp_images directory using the format PRENOM_NOM_DDMMYY-HH.jpg



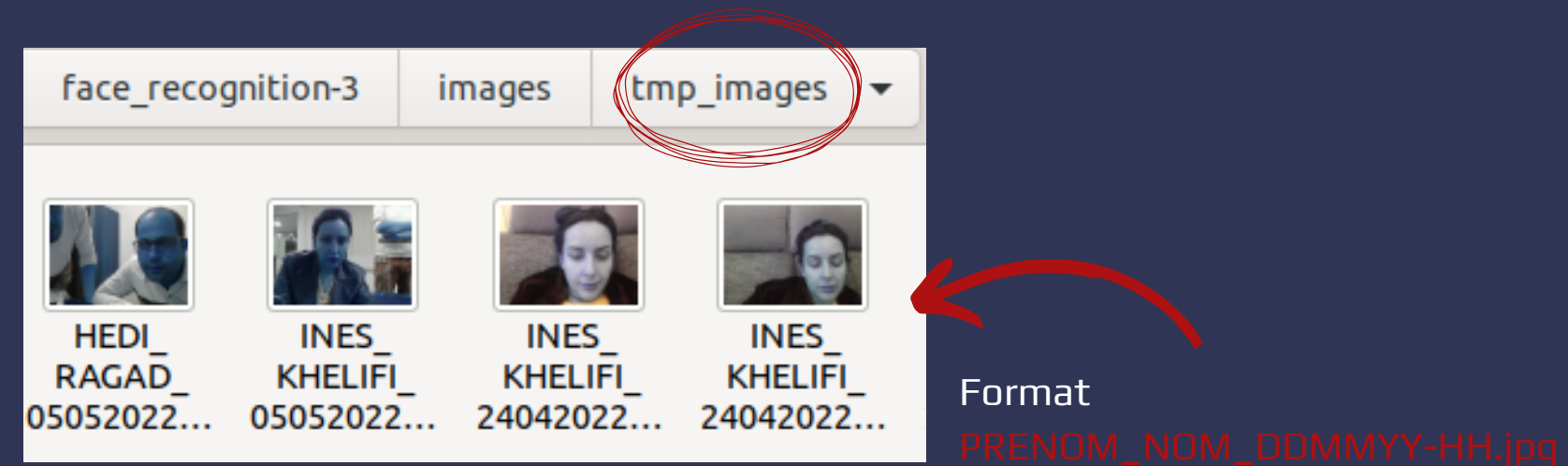
For each picture provided in the database we produce using data augmentation (zoom, rotation etc ...) 10 different pictures saved in images/keras directory



Results

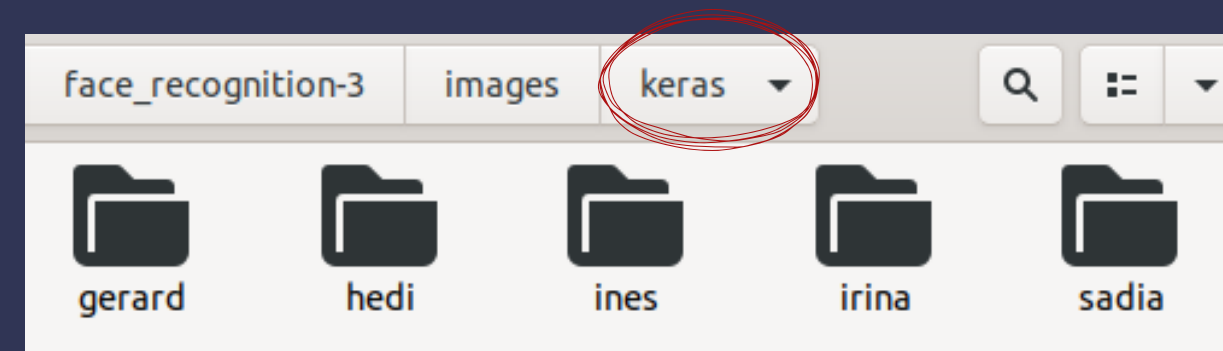
1-Test data

Are the images identified in real time using cv2 and face_recognition library and can be found in images/tmp_images directory.



2-Train data

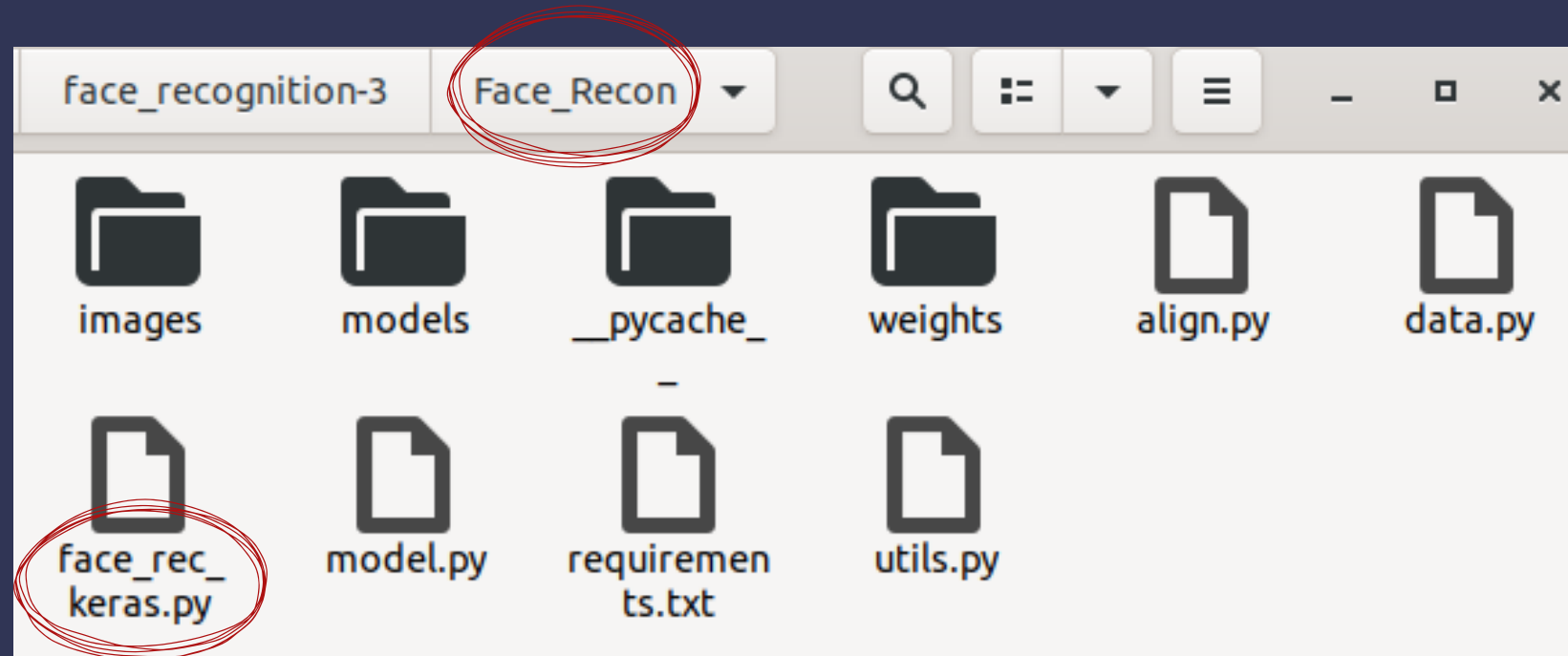
Issued from keras data augmentation and can be found in images/keras directory



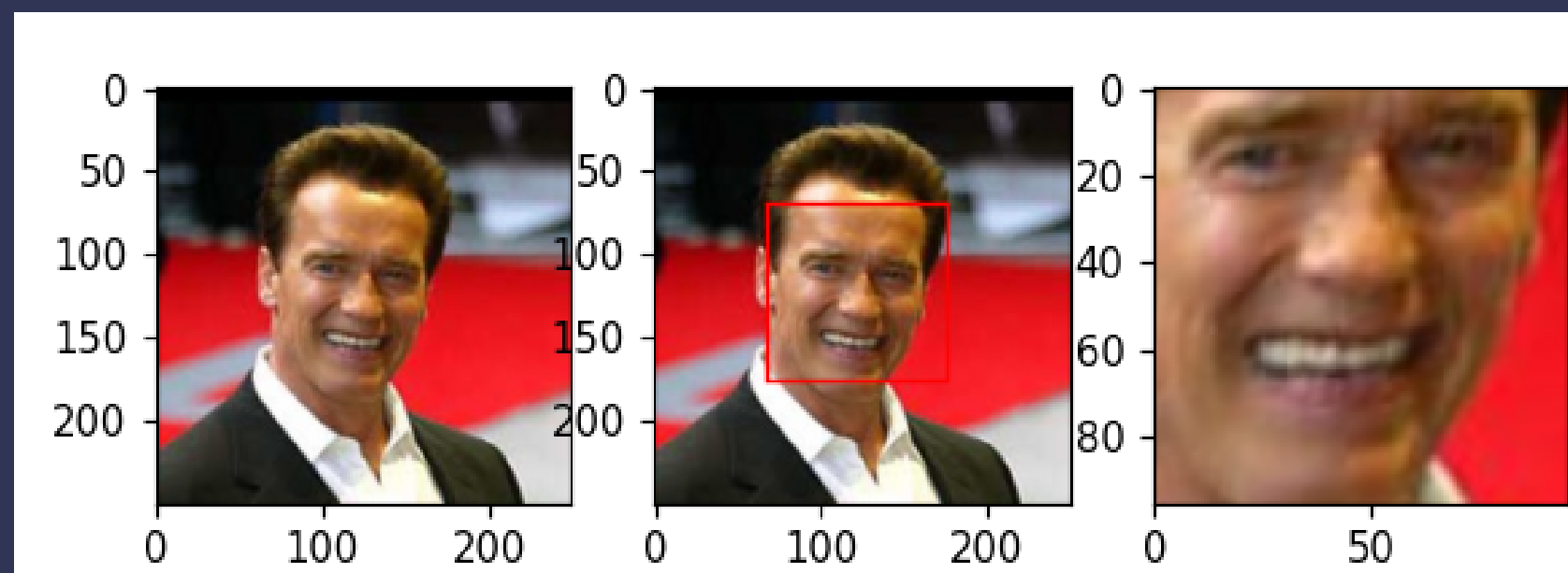
Deep Face Recognition with Keras, Dlib and OpenCV



Face Alignment



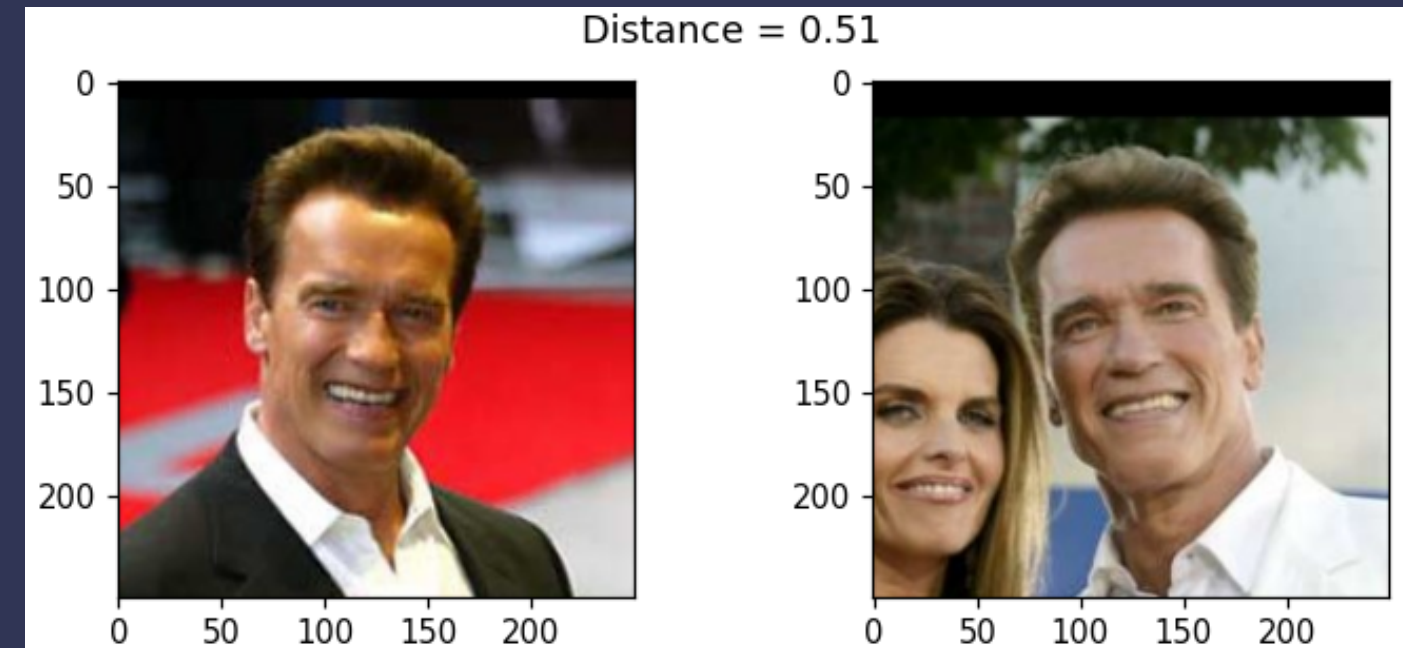
The **nn4.small2.v1** model was trained with aligned face images, therefore, the face images from the custom dataset must be aligned too. Here, we use **Dlib** for face detection and **OpenCV** for image transformation and cropping to produce aligned 96x96 RGB face images.



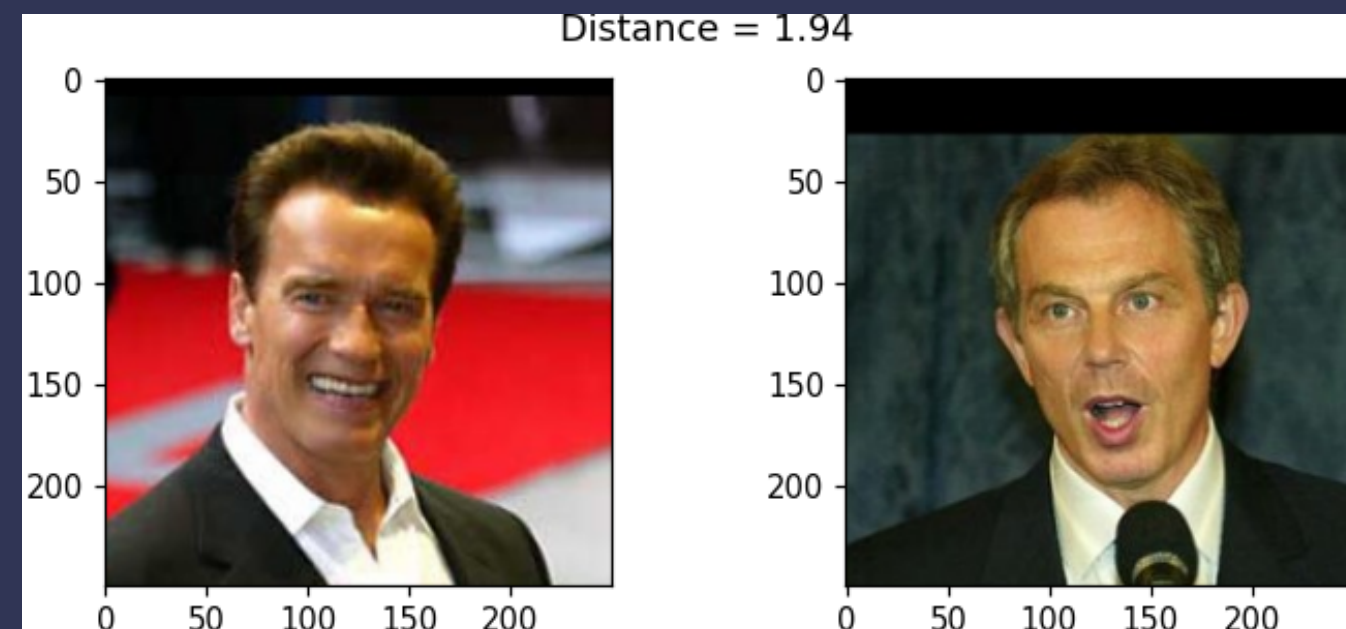
- 1- Image Resizing
- 2- Face detection
- 3- Image zoom and cropping

Embedding Vectors

```
def show_pair(idx1, idx2):  
    plt.figure(figsize=(8,3))  
    plt.suptitle(f'Distance = {distance(embedded[idx1], e  
    plt.subplot(121)  
    plt.imshow(load_image(metadata[idx1].image_path()))  
    plt.subplot(122)  
    plt.imshow(load_image(metadata[idx2].image_path()));  
    plt.show()  
    return  
  
show_pair(15, 16)  
show_pair(15, -12)
```



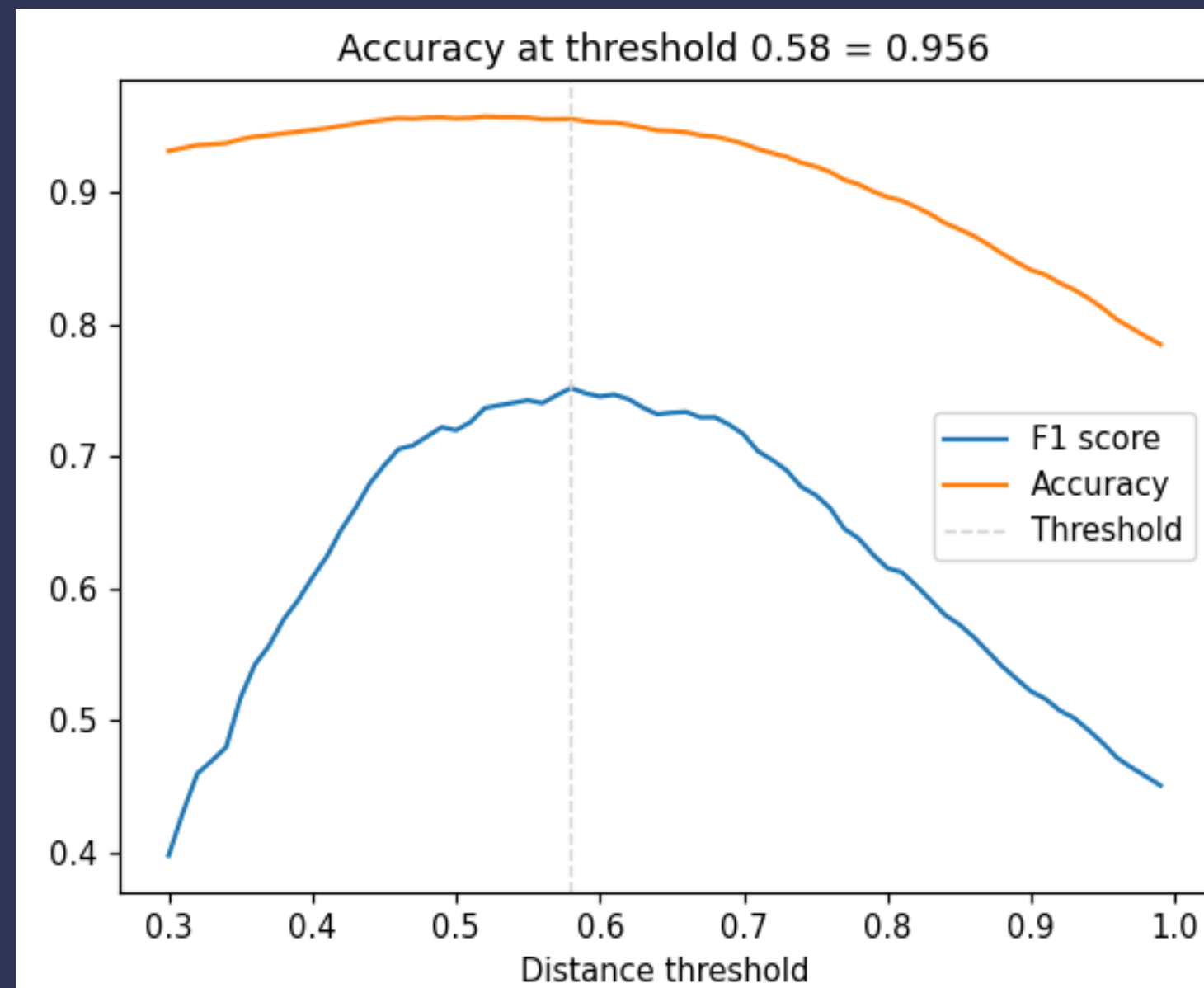
Let's verify on a single triplet example that the squared L2 distance between its anchor-positive pair is smaller than the distance between its anchor-negative pair.



- **Distance** between photo 15 and 16 in the database is **0.51 < 1** so it is likely to be the same person.
- **Distance** between photo 15 and 12 in the database is **1.94 > 1** so it is not the same person.

Distance threshold

All possible embedding vector pairs are classified as either same identity or different identity and compared to the ground truth. Since we're dealing with skewed classes (much more negative pairs than positive pairs), we use the F1 score as evaluation metric instead of accuracy.



$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$F_1 = \frac{2}{\text{recall}^{-1} + \text{precision}^{-1}} = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{tp}{tp + \frac{1}{2}(fp + fn)}$$

$$\text{Precision} = \frac{tp}{tp + fp}$$
$$\text{Recall} = \frac{tp}{tp + fn}$$

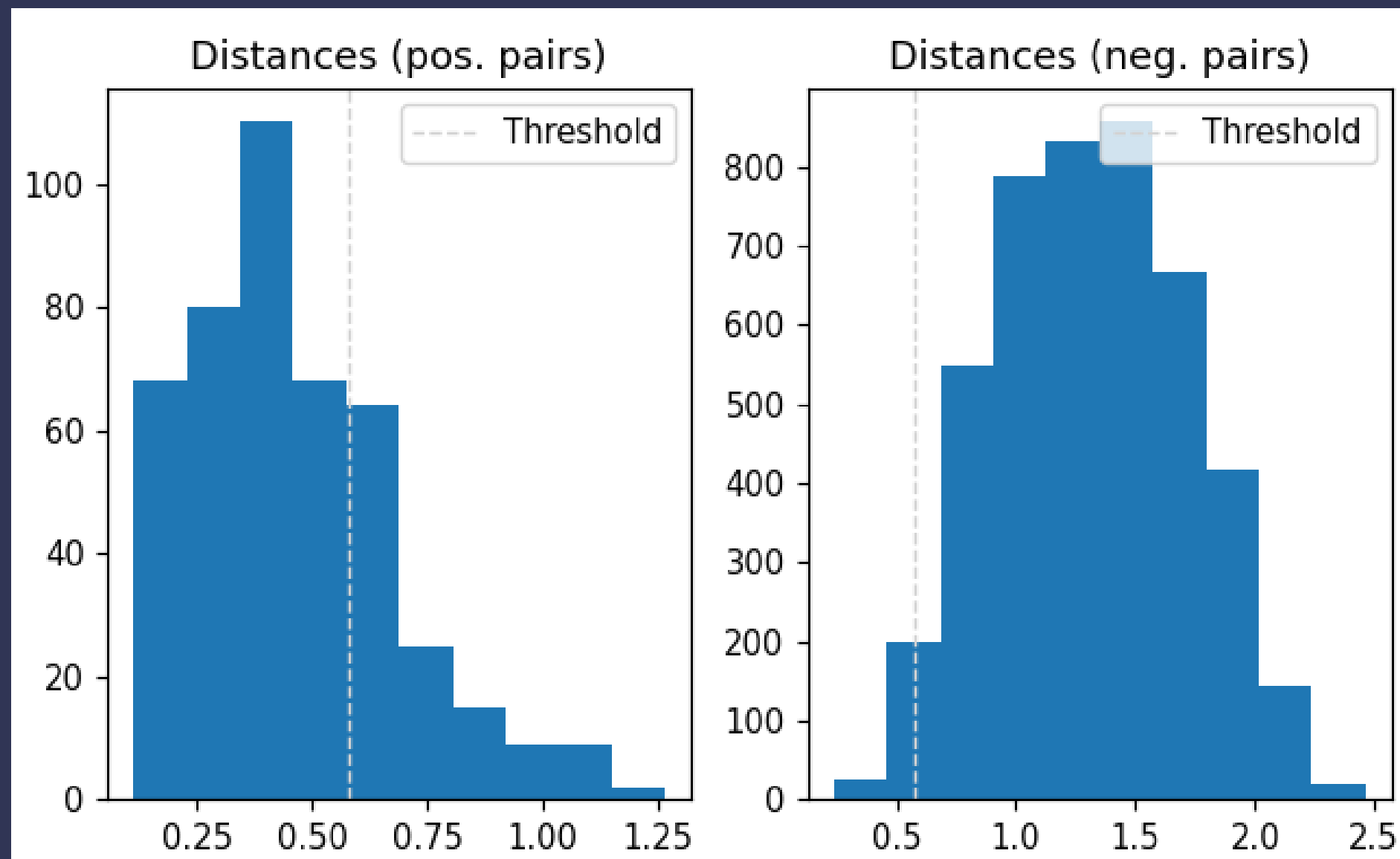
References : <https://en.wikipedia.org/wiki/F-score>

https://en.wikipedia.org/wiki/Precision_and_recall#Precision

https://en.wikipedia.org/wiki/Accuracy_and_precision#In_binary_classification

Histogram of distance pairs

The face verification accuracy at $\tau = 0.58$ is 95.6%. This is not bad given a baseline of 89% for a classifier that always predicts different identity (there are 450 pos. pairs and 4500 neg. pairs) but since nn4.small2.v1 is a relatively small model it is still less than what can be achieved by state-of-the-art models ($> 99\%$).



```
dist_pos = distances[identical == 1]
dist_neg = distances[identical == 0]

plt.figure(figsize=(12,4))

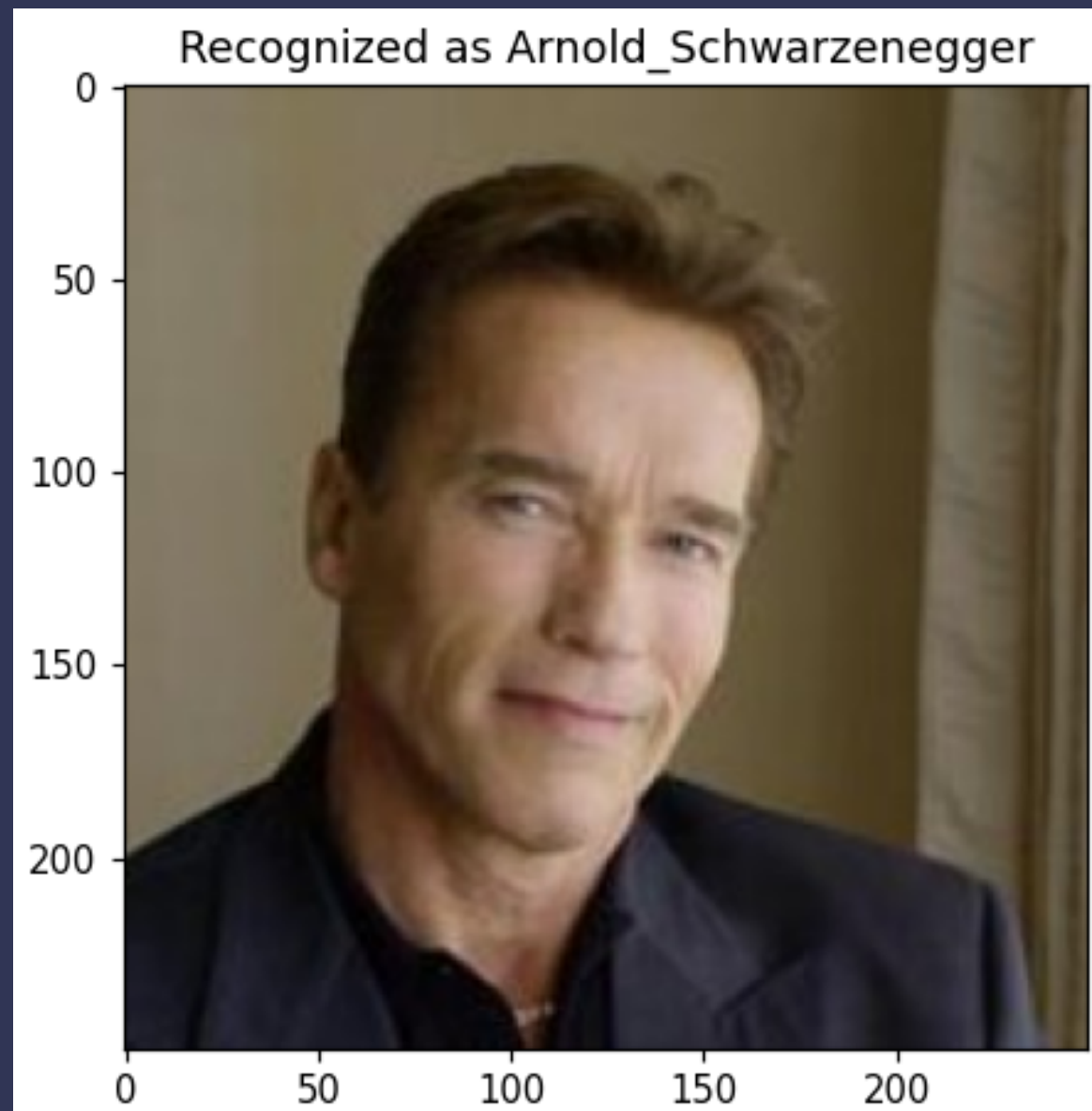
plt.subplot(121)
plt.hist(dist_pos)
plt.axvline(x=opt_tau, linestyle='--', lw=1,
plt.title('Distances (pos. pairs)')
plt.legend();

plt.subplot(122)
plt.hist(dist_neg)
plt.axvline(x=opt_tau, linestyle='--', lw=1,
plt.title('Distances (neg. pairs)')
plt.legend();
```

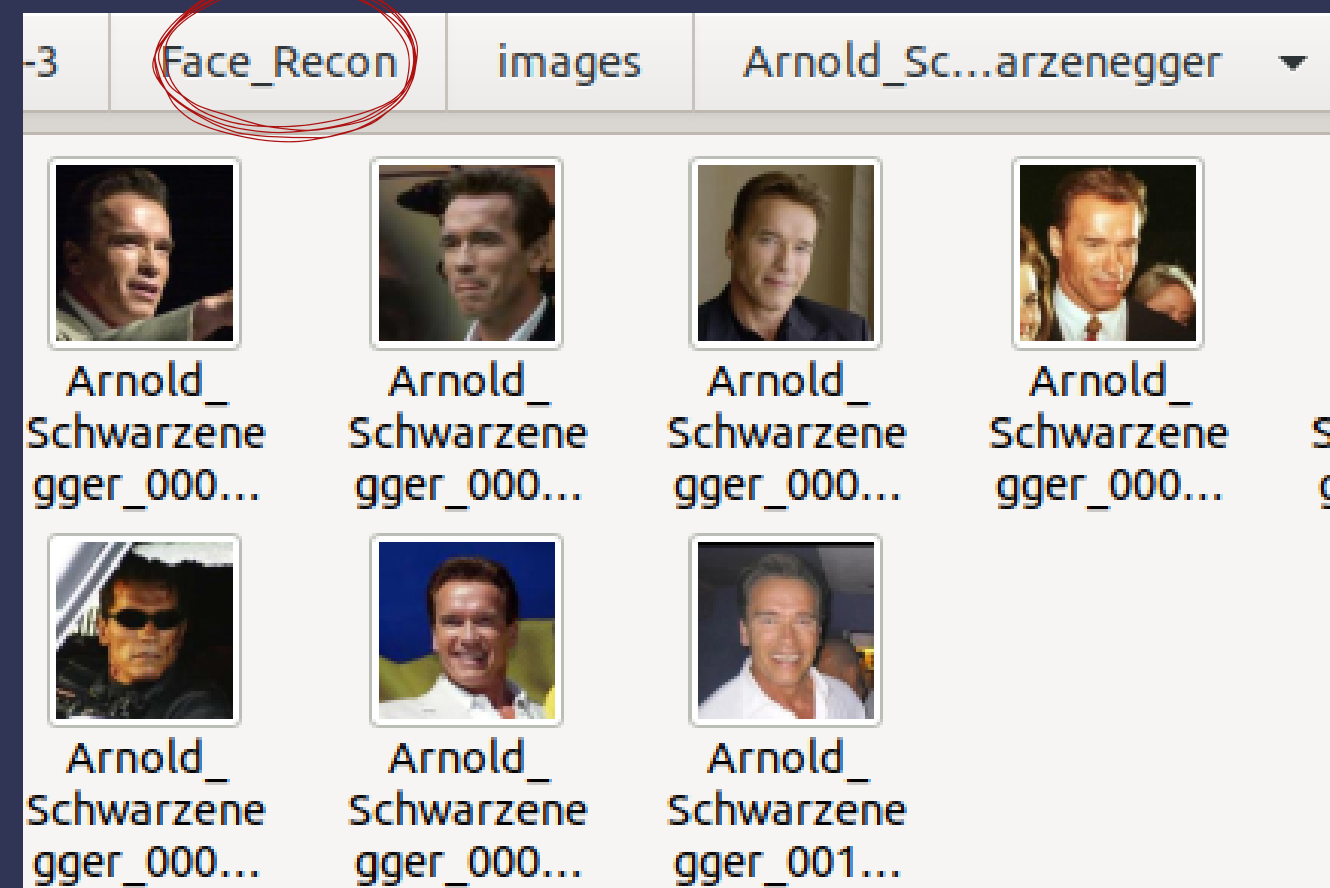
The following two histograms show the distance distributions of positive and negative pairs and the location of the decision boundary. There is a clear separation of these distributions which explains the discriminative performance of the network. One can also spot some strong outliers in the positive pairs class but these are not further analyzed here.

Face Recognition

A more robust approach is to label the input using the top k scoring entries in the database which is essentially **KNN classification** with a Euclidean distance metric. Alternatively, a **linear support vector machine (SVM)** can be trained with the database entries and used to classify i.e. identify new inputs. For training these classifiers we use 50% of the dataset, for evaluation the other 50%.

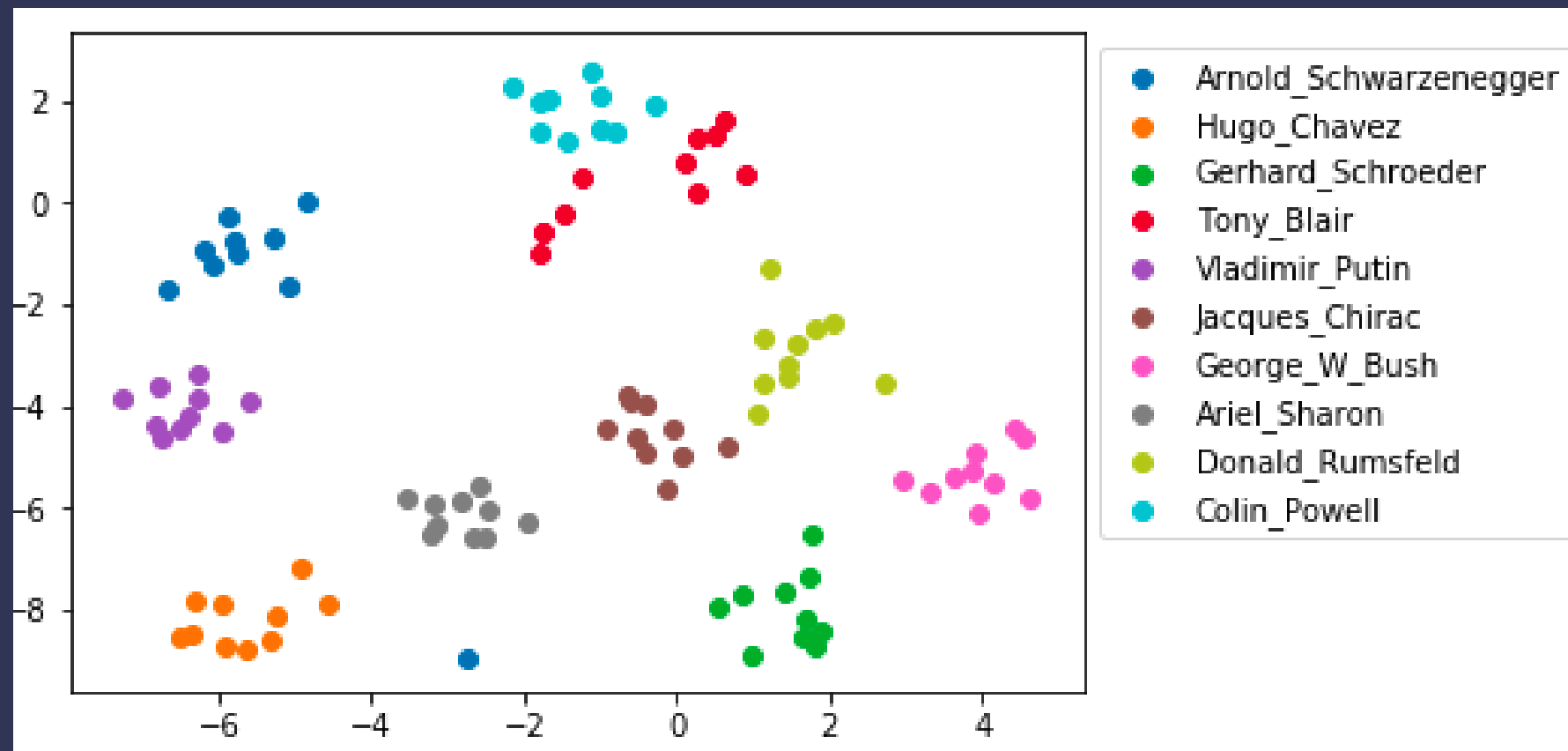


The training data is in
Face_Recon/images directory



Dataset visualization

To embed the dataset into 2D space for displaying identity clusters, t-distributed Stochastic Neighbor Embedding (t-SNE) is applied to the 128-dimensional embedding vectors. Except from a few outliers, identity clusters are well separated.



```
targets = np.array([m.name for m in metadata])
```

```
from sklearn.manifold import TSNE

def graph4():
    X_embedded = TSNE(n_components=2).fit_transform(embedded)

    for i, t in enumerate(set(targets)):
        idx = targets == t
        plt.scatter(X_embedded[idx, 0], X_embedded[idx, 1], label=t)

    plt.legend(bbox_to_anchor=(1, 1));
    plt.show()
    return
```

Thank you for your attention